

Quantum Time Evolution through Trotterization

The goal of this project is to simulate quantum dynamics or time evolution of lattice systems described by the Ising model Hamiltonian (https://en.wikipedia.org/wiki/Ising_model). Compared to classical computers, quantum computers have a natural advantage in representing the unitary operators that generate time evolution, making it a natural laboratory.

In quantum mechanics, a particle is represented by a 'ket' state $|\psi(t)\rangle$. Its inner product with a 'bra' $\langle x|\psi\rangle \equiv \psi(x)$, known as the wavefunction of the particle, is interpreted as the probability amplitude of finding the particle at location x upon measurement. The particle's motion in time is governed by the time-dependent Schrodinger's equation:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle$$

where $\hat{H}$ is a Hermitian operator known as the Hamiltonian. The solution to this equation is simply

$$|\psi(t)\rangle = e^{i\hat{H}t} |\psi(0)\rangle$$

where we adopt the natural unit system in which $\hbar = 1$. If the Hamiltonian or energy operator can be written as a sum of Pauli operators $\hat{H} = \sum_i a_i \hat{H}_i$ (as in the case of the Ising model, for example), then to a good first-order approximation,

$$|\psi(t)\rangle = e^{i\hat{H}t} |\psi(0)\rangle \approx \left(\prod_j e^{ia_j \hat{H}_j \Delta t} \right)^{\frac{t}{\Delta t}} |\psi(0)\rangle.$$

This is known as trotterization, which allows us to approximate (very well, if we choose small enough Δt) the exponential of noncommutative matrix operators in the above form. This creates a quantum gate to be applied $t/\Delta t$ times. For each time interval Δt , the unitary operator $U(t) = e^{-i\Delta t G}$ for Hermitian $G = a_j \hat{H}_j$ can be readily implemented using the PauliEvolutionGate. For our specific project, we consider the familiar Ising model Hamiltonian from class, with one term (J) describing the interaction energy and another (h) describing an external field strength:

$$\hat{H} = -J \sum_{i=0}^{L-2} \sigma_i \sigma_{i+1} - h \sum_{i=0}^{L-1} \sigma_i$$

We will focus on the spin along the z -direction. It is also convenient for later to parametrize the second term describing the field in terms of an orientation/angle α :

$$\hat{H} = -J \sum_{i=0}^{L-2} Z_i Z_{i+1} - h \sum_{i=0}^{L-1} (\sin \alpha Z_i + \cos \alpha X_i).$$

Recall that for Pauli operators $Q \in \{X, Y, Z\}$, each time evolution step is simply realized by a rotation gate $R_Q(t) = e^{-i\frac{t}{2}Q} = \cos \frac{t}{2}I - i \sin \frac{t}{2}Q$. We will, however, use the more general Trotter Quantum Real Time Evolution (TrotterQRTE) gate for this project.

```
In [24]: from qiskit.quantum_info import SparsePauliOp
from math import sin, cos

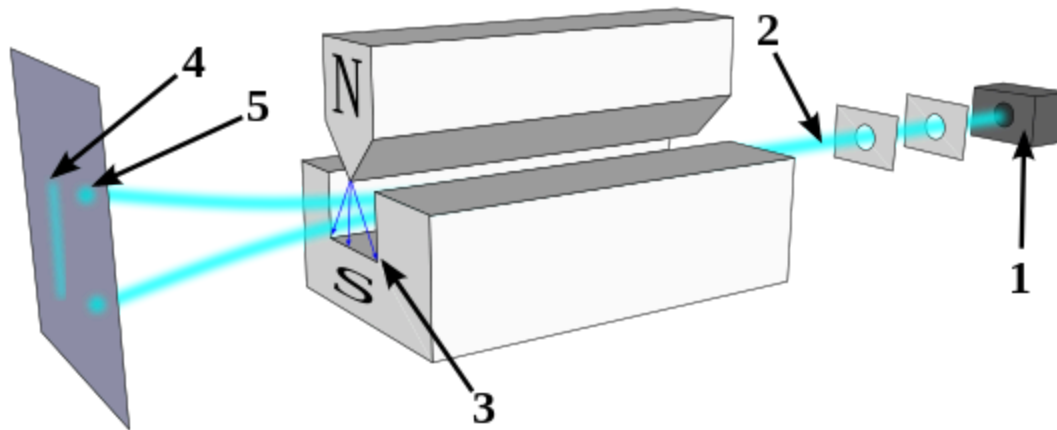
# Function for defining our Hamiltonian, with parameters
# (1) L, the number of lattice sites;
# (2) J, the interaction energy term;
# (3) h, the field strength term;
# (4) alpha, the angle inclination of the field, with alpha = 90 being parallel and alpha = 0 being perpendicular
def get_hamiltonian(L, J, h, alpha=0):

    ZZ_tuples = [("ZZ", [i, i + 1], -J) for i in range(0, L - 1)]
    Z_tuples = [("Z", [i], -h * sin(alpha)) for i in range(0, L)]
    X_tuples = [("X", [i], -h * cos(alpha)) for i in range(0, L)]

    hamiltonian = SparsePauliOp.from_sparse_list([*ZZ_tuples, *Z_tuples, *X_tuples], num_qubits=L)
    return hamiltonian.simplify()
```

Stern-Gerlach Experiment

Let's perform a preliminary example in which a single fermion (such as an electron) is subject to an external magnetic field and observe how its spin evolves. This is a simulation of the classic Stern-Gerlach experiment (https://en.wikipedia.org/wiki/Stern%E2%80%93Gerlach_experiment). From quantum physics, we know the particle's spin is only affected by magnetic field transverse to its spin, while field that is parallel/longitudinal exerts no effect.



In this case, we do not have any interaction energy term and simply consider the Hamiltonian of an electron in a magnetic field

$$H = g\mu_b \vec{S} \cdot \vec{B}.$$

```
In [25]: from math import pi
```

```
H = get_hamiltonian(L=1, J=0, h=1.0, alpha=pi / 8)
H
```

```
Out[25]: SparsePauliOp(['Z', 'X'],
                        coeffs=[-0.38268343+0.j, -0.92387953+0.j])
```

```
In [26]: from qiskit.quantum_info import Statevector
         from qiskit_algorithms import TimeEvolutionProblem
```

```
final_time = 1.6
initial_state = Statevector.from_label("0")
problem = TimeEvolutionProblem(H, initial_state=initial_state, time=final_time)
```

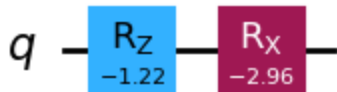
We make use of the Trotter Quantum Real Time Evolution (TrotterQRTE) to do this

```
In [27]: from qiskit_algorithms import TrotterQRTE
```

```
trotter = TrotterQRTE()
result = trotter.evolve(problem)
```

```
In [28]: result.evolved_state.decompose(reps=2).decompose("disentangler_dg").decompose(
    "multiplex1_reverse_dg"
).draw("mpl")
```

Out[28]:



It is in fact just two rotation gates, as expected.

```
In [31]: import numpy as np
import matplotlib.pyplot as plt
```

```
bar_width = 0.1
```

```
initial_state = Statevector.from_label("0")
trotter = TrotterQRTE()
final_time = 0.5
eps = 1e-5
```

```
alphas = np.linspace(-np.pi / 2 + eps, np.pi / 2 - eps, 5)
```

```
for i, alpha in enumerate(alphas):
    H_alpha = get_hamiltonian(L=1, J=0, h=1.0, alpha=alpha)
    problem = TimeEvolutionProblem(H_alpha, initial_state=initial_state, time=final_time)
    result = trotter.evolve(problem)
```

```

evolved_state = Statevector(result.evolved_state)

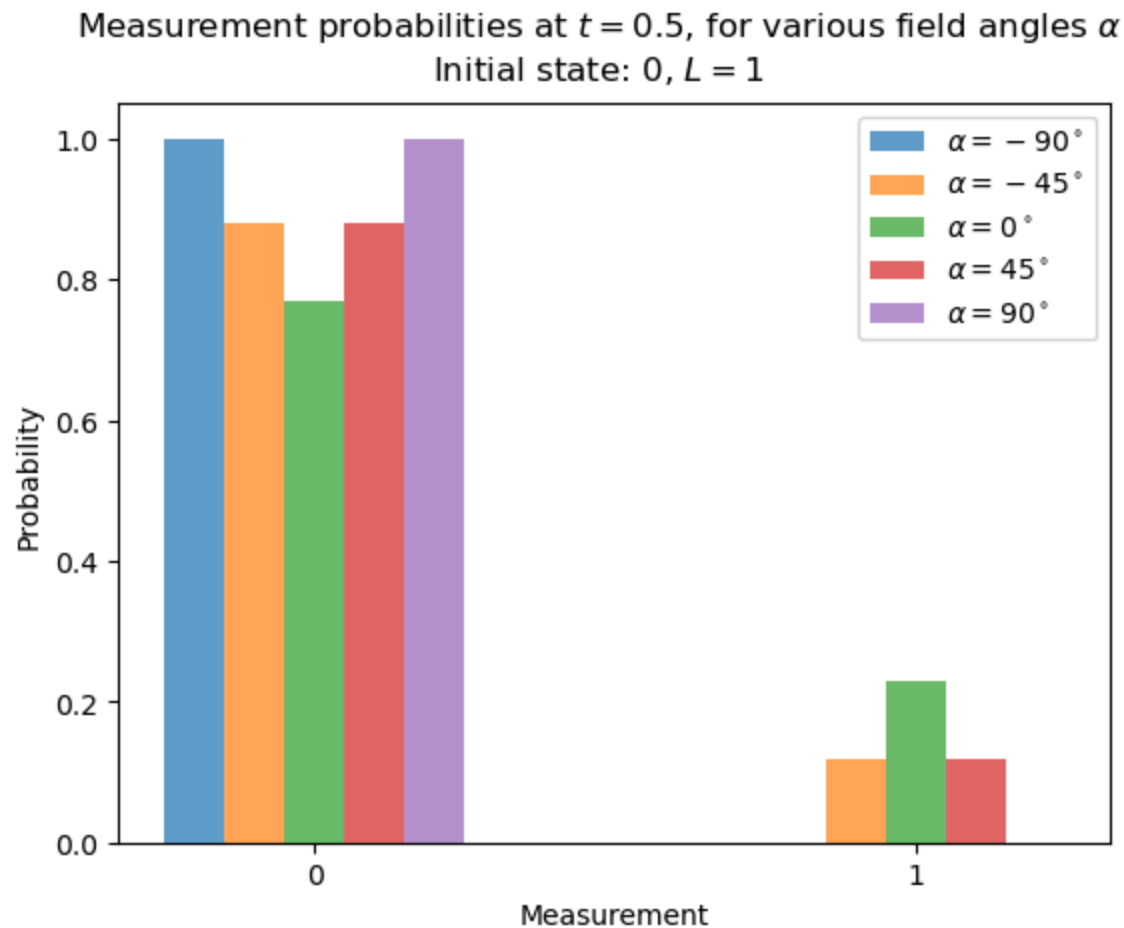
amplitudes_dict = evolved_state.probabilities_dict()
labels = list(amplitudes_dict.keys())
values = list(amplitudes_dict.values())

alpha_str = f"$\\alpha={int(np.round(alpha * 180 / pi))}^\\circ$"
plt.bar(np.arange(2) + i * bar_width, values, bar_width, label=alpha_str, alpha=0.7)

plt.xticks(np.arange(2) + 2 * bar_width, labels)
plt.xlabel("Measurement")
plt.ylabel("Probability")
plt.suptitle(
    f"Measurement probabilities at $t={final_time}$, for various field angles $\\alpha$\\n"
    f"Initial state: 0, $L=1$"
)
plt.legend()

```

Out[31]: <matplotlib.legend.Legend at 0x16a171850>



We observe that when the field is completely longitudinal ($\alpha = \pm 90^\circ$), the spin as expected has not changed, while the spin flips when the field is completely transverse ($\alpha = 0^\circ$). We see spin flips more often than not when the field is pointed somewhere in between.

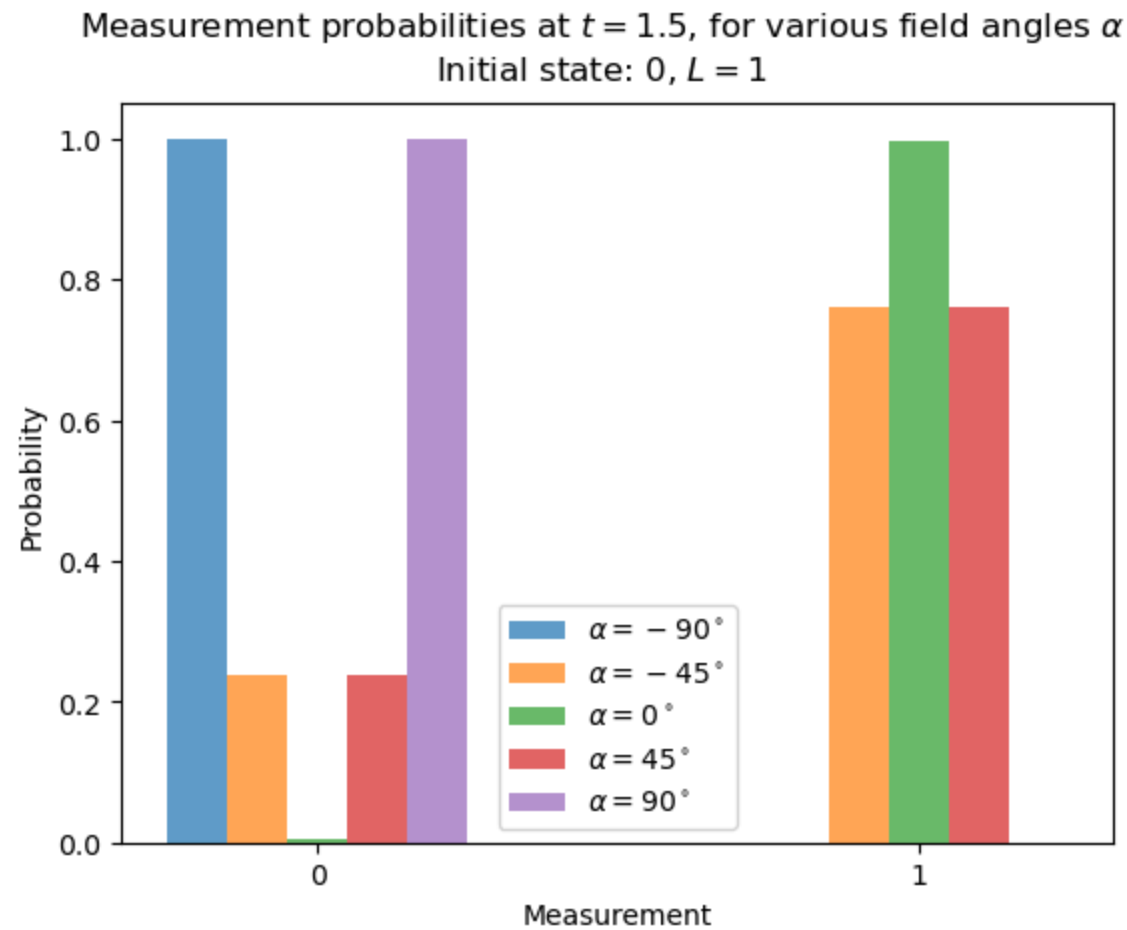
```
In [32]: final_time = 1.5
for i, alpha in enumerate(alphas):
    H_alpha = get_hamiltonian(L=1, J=0.2, h=1.0, alpha=alpha)
    problem = TimeEvolutionProblem(H_alpha, initial_state=initial_state, time=final_time)
    result = trotter.evolve(problem)
    evolved_state = Statevector(result.evolved_state)
```

```
amplitudes_dict = evolved_state.probabilities_dict()
labels = list(amplitudes_dict.keys())
values = list(amplitudes_dict.values())

alpha_str = f"$\\alpha={int(np.round(alpha * 180 / pi))}^\\circ$"
plt.bar(np.arange(2) + i * bar_width, values, bar_width, label=alpha_str, alpha=0.7)

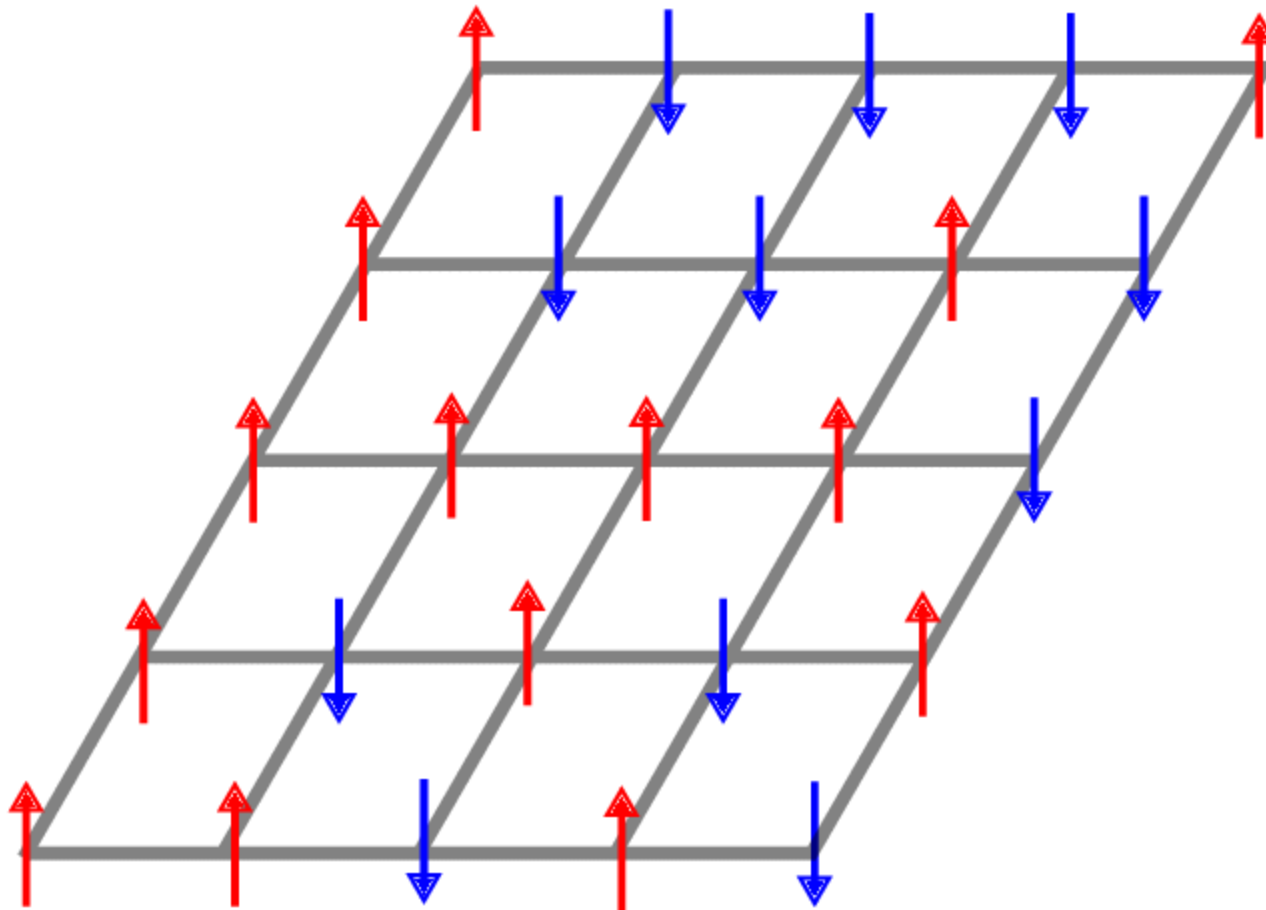
plt.xticks(np.arange(2) + 2 * bar_width, labels)
plt.xlabel("Measurement")
plt.ylabel("Probability")
plt.suptitle(
    f"Measurement probabilities at $t={final_time}$, for various field angles $\\alpha$\n"
    f"Initial state: 0, $L=1$"
)
plt.legend()
```

Out[32]: <matplotlib.legend.Legend at 0x16a1e5510>



Lattice

Now we move on to consider $L = 6$ sites in our lattice, which will incorporate interaction energy.



In [34]: `from math import pi`

`L = 6`

`H = get_hamiltonian(L=L, J=0.3, h=1, alpha=pi / 8)`

In [35]: `from qiskit_algorithms import TrotterQ RTE`

`from qiskit.primitives import Estimator`

`num_timesteps = 60`

`trotter = TrotterQ RTE(num_timesteps=num_timesteps, estimator=Estimator())`

We define two observables, magnetization $M = \sum_{i=1}^L Z_i$ and spin correlation $S = \sum_i Z_i Z_{i+1} / (L - 1)$. Over time, we expect magnetization to approach 0 as the spins align themselves.

```
In [36]: magnetization_op = SparsePauliOp.from_sparse_list(
    [ ("Z", [i], 1.0) for i in range(0, L)], num_qubits=L
)
correlation_op = SparsePauliOp.from_sparse_list(
    [ ("ZZ", [i, i + 1], 1.0) for i in range(0, L - 1)], num_qubits=L
) / (L - 1)
```

Initialization of our wavefunction $|\psi_0\rangle = |001001\rangle$.

```
In [37]: final_time = 60.0
initial_state = Statevector.from_label("001001")
problem = TimeEvolutionProblem(
    H,
    initial_state=initial_state,
    time=final_time,
    aux_operators=[H, magnetization_op, correlation_op],
)
```

```
In [38]: result = trotter.evolve(problem)
```

```
In [39]: import numpy as np

observables = np.array(np.array(result.observables)[: , : , 0])
observables.shape
```

```
Out[39]: (61, 3)
```

```
In [40]: import matplotlib.pyplot as plt

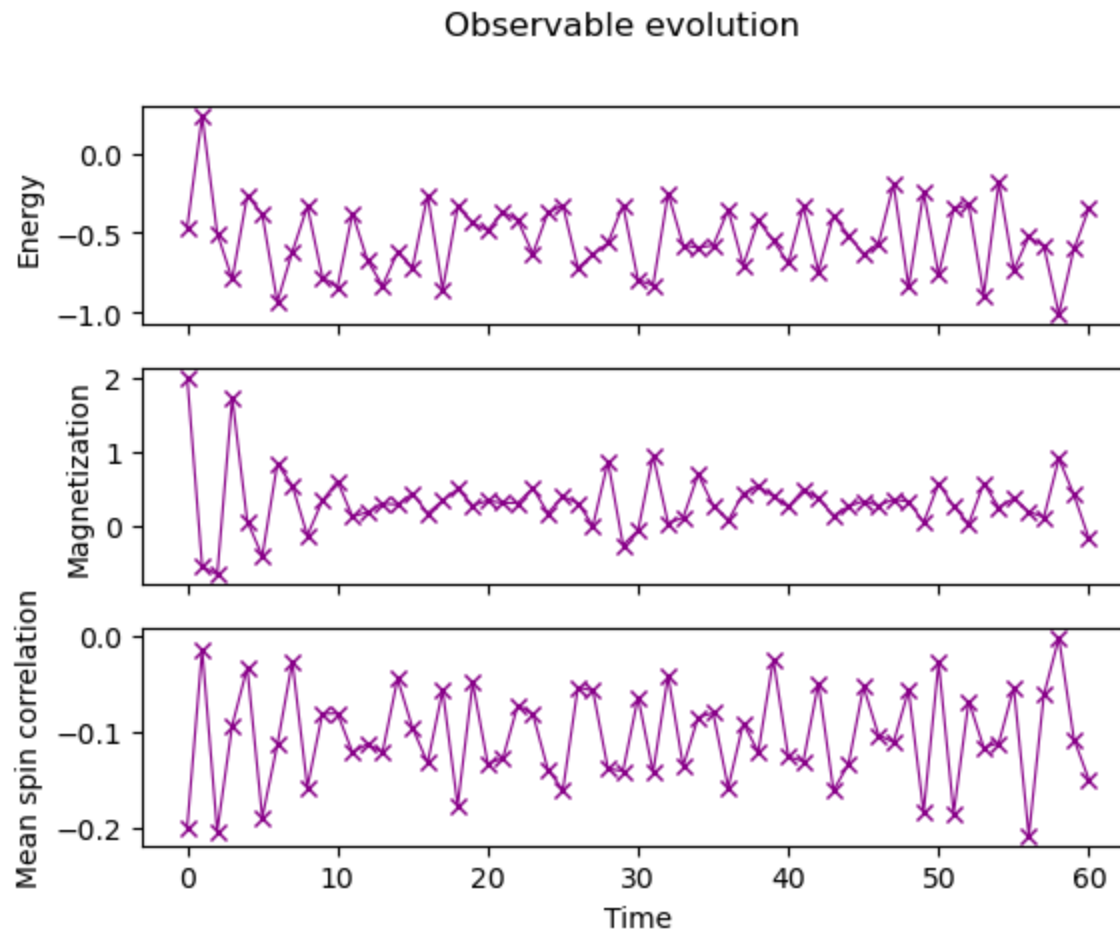
fig, axes = plt.subplots(3, sharex=True)
times = np.linspace(0, final_time, num_timesteps + 1) # includes initial state
axes[0].plot(
    times, observables[:, 0], label="First order", marker="x", c="darkmagenta", ls="--", lw=0.8
)
axes[1].plot(
    times, observables[:, 1], label="First order", marker="x", c="darkmagenta", ls="--", lw=0.8
```

```

)
axes[2].plot(
    times, observables[:, 2], label="First order", marker="x", c="darkmagenta", ls="-", lw=0.8
)
axes[0].set_ylabel("Energy")
axes[1].set_ylabel("Magnetization")
axes[2].set_ylabel("Mean spin correlation")
axes[2].set_xlabel("Time")
fig.suptitle("Observable evolution")

```

Out[40]: Text(0.5, 0.98, 'Observable evolution')



We see that the energy fluctuates over time due to low accuracy, but the magnetization does approach a constant value. Let's do an analytic solution using scipy package.

In [41]: `import scipy as sc`

`H_array = H.to_matrix()`

We define a slightly denser time mesh

`exact_times = np.linspace(0, final_time, 101)`

We compute the exact evolution using the exp

```
exact_evolution = [
    initial_state.evolve(sc.linalg.expm(-1j * time * H_array)) for time in exact_times
]
```

In [42]: `exact_energy = np.real([sv.expectation_value(H) for sv in exact_evolution])`

`exact_magnetization = np.real([sv.expectation_value(magnetization_op) for sv in exact_evolution])`

`exact_correlation = np.real([sv.expectation_value(correlation_op) for sv in exact_evolution])`

In [43]: `axes[0].plot(exact_times, exact_energy, c="k", ls=":", label="Exact")`

`axes[1].plot(exact_times, exact_magnetization, c="k", ls=":", label="Exact")`

`axes[2].plot(exact_times, exact_correlation, c="k", ls=":", label="Exact")`

Select the labels of only the first axis

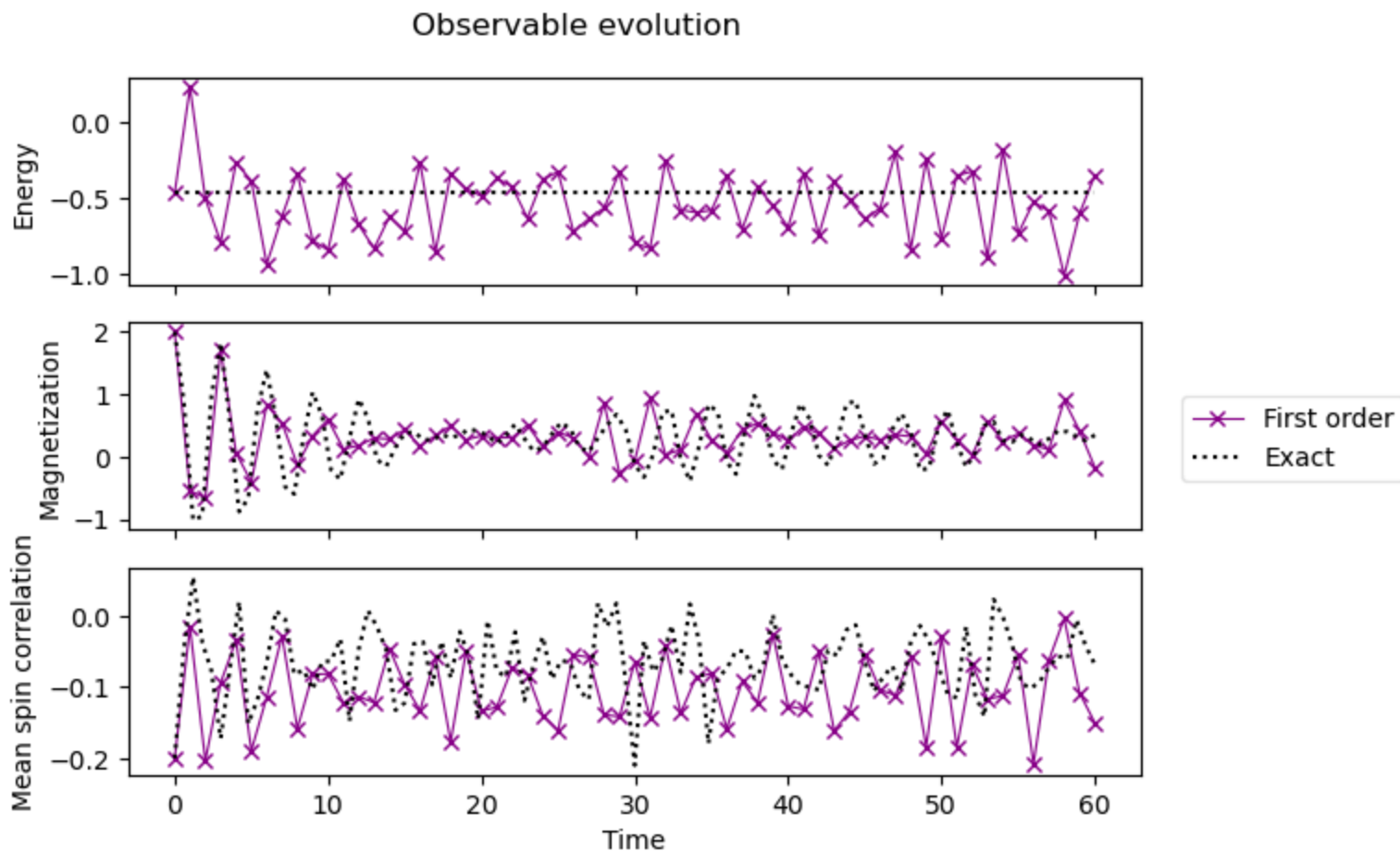
```
legend = fig.legend(
    *axes[0].get_legend_handles_labels(),
    bbox_to_anchor=(1.0, 0.5),
    loc="center left",
    framealpha=0.5,
```

)

`fig.tight_layout()`

`fig`

Out[43]:



The code we have implemented so far represents first-order formulas that, as we can see, deviate quite noticeably from the exact solution. We can incorporate higher-order corrections that improve accuracy.

```
In [44]: from qiskit.circuit import QuantumCircuit
from qiskit.circuit.library import PauliEvolutionGate
from qiskit.synthesis import LieTrotter

dt = final_time / num_timesteps
trotter_step_first_order = PauliEvolutionGate(H, dt, synthesis=LieTrotter())

# We create an empty circuit
circuit = QuantumCircuit(H.num_qubits)
```

```

circuit.append(trotter_step_first_order, range(H.num_qubits))
circuit = circuit.decompose(reps=2)

# Let us print some stats
print(
    f"""
    Trotter step with Lie-Trotter
    -----

            Depth: {circuit.depth()}
            Gate count: {len(circuit)}
            Nonlocal gate count: {circuit.num_nonlocal_gates()}
            Gate breakdown: {"", ".join([f"{k.upper()}: {v}" for k, v in circuit.count_ops().items()])}
    """,
)

# And finally draw the circuit
circuit.draw("mpl")

```

Trotter step with Lie-Trotter

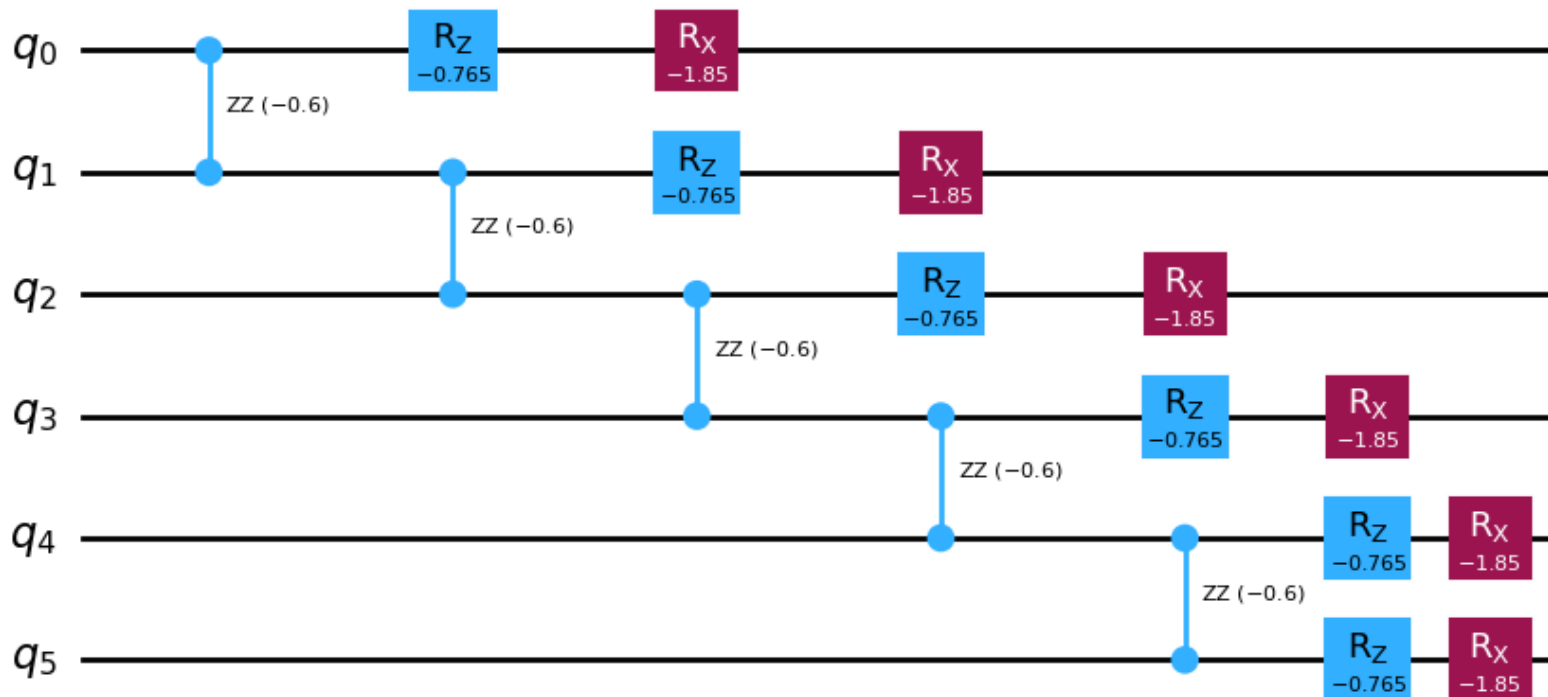
```

-----

            Depth: 7
            Gate count: 17
            Nonlocal gate count: 5
            Gate breakdown: RZ: 6, RX: 6, RZZ: 5

```

Out[44]:



The higher-order formulas can be realized using the Suzuki-Trotter formula, which is a second-order decomposition:

$$e^{A+B} \approx e^{B/2} e^A e^{B/2}$$

Higher-order approximations can be done with recursion and are simply implemented with Qiskit's SuzukiTrotter class.

```
In [45]: from qiskit.synthesis import SuzukiTrotter

second_order_formula = SuzukiTrotter() # if not passed, order defaults to 2
trotter_step_second_order = PauliEvolutionGate(H, dt, synthesis=second_order_formula)
circuit = QuantumCircuit(H.num_qubits)
circuit.append(trotter_step_second_order, range(H.num_qubits))
circuit = circuit.decompose(reps=2)

# Let us print some stats
print()
```

```

f"""
Trotter step with Suzuki Trotter (2nd order)
-----

        Depth: {circuit.depth()}
        Gate count: {len(circuit)}
        Nonlocal gate count: {circuit.num_nonlocal_gates()}
        Gate breakdown: {", ".join([f"{k.upper()}: {v}" for k, v in circuit.count_ops().items()])}

"""
)

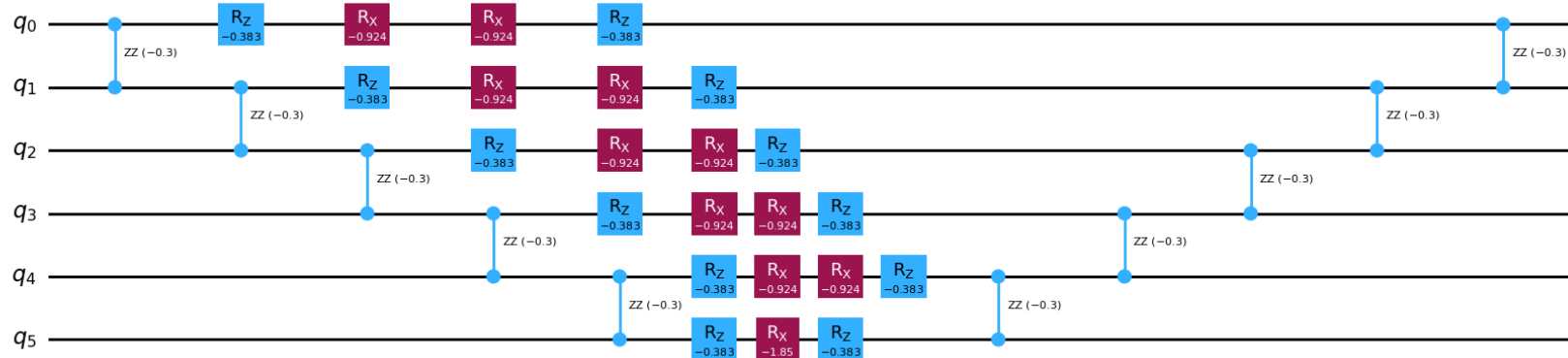
# And finall
circuit.draw("mpl")

```

Trotter step with Suzuki Trotter (2nd order)

Depth: 14
Gate count: 33
Nonlocal gate count: 10
Gate breakdown: RZ: 12, RX: 11, RZZ: 10

Out[45]:



```

In [46]: sixth_order_formula = SuzukiTrotter(order=6)
trotter_step_sixth_order = PauliEvolutionGate(H, dt, synthesis=sixth_order_formula)
circuit = QuantumCircuit(H.num_qubits)
circuit.append(trotter_step_sixth_order, range(H.num_qubits))

```



```

circuit = circuit.decompose(reps=2)

# Let us print some stats
print(
    f"""
    Trotter step with Suzuki Trotter (6th order)
    -----

            Depth: {circuit.depth()}
            Gate count: {len(circuit)}
            Nonlocal gate count: {circuit.num_nonlocal_gates()}
            Gate breakdown: {"", ".join([f"{k.upper()}: {v}" for k, v in circuit.count_ops().items()])}

    """
)

```

Trotter step with Suzuki Trotter (6th order)

```

-----

            Depth: 350
            Gate count: 825
            Nonlocal gate count: 250
            Gate breakdown: RZ: 300, RX: 275, RZZ: 250

```

In [47]: `from qiskit.synthesis import SuzukiTrotter`

```

trotter = TrotterQRTE(SuzukiTrotter(order=6), num_timesteps=num_timesteps, estimator=Estimator())
problem = TimeEvolutionProblem(
    H,
    initial_state=initial_state,
    time=final_time,
    aux_operators=[H, magnetization_op, correlation_op],
)
result = trotter.evolve(problem)
observables_order6 = np.array(np.array(result.observables)[: , : , 0], dtype=np.float64)

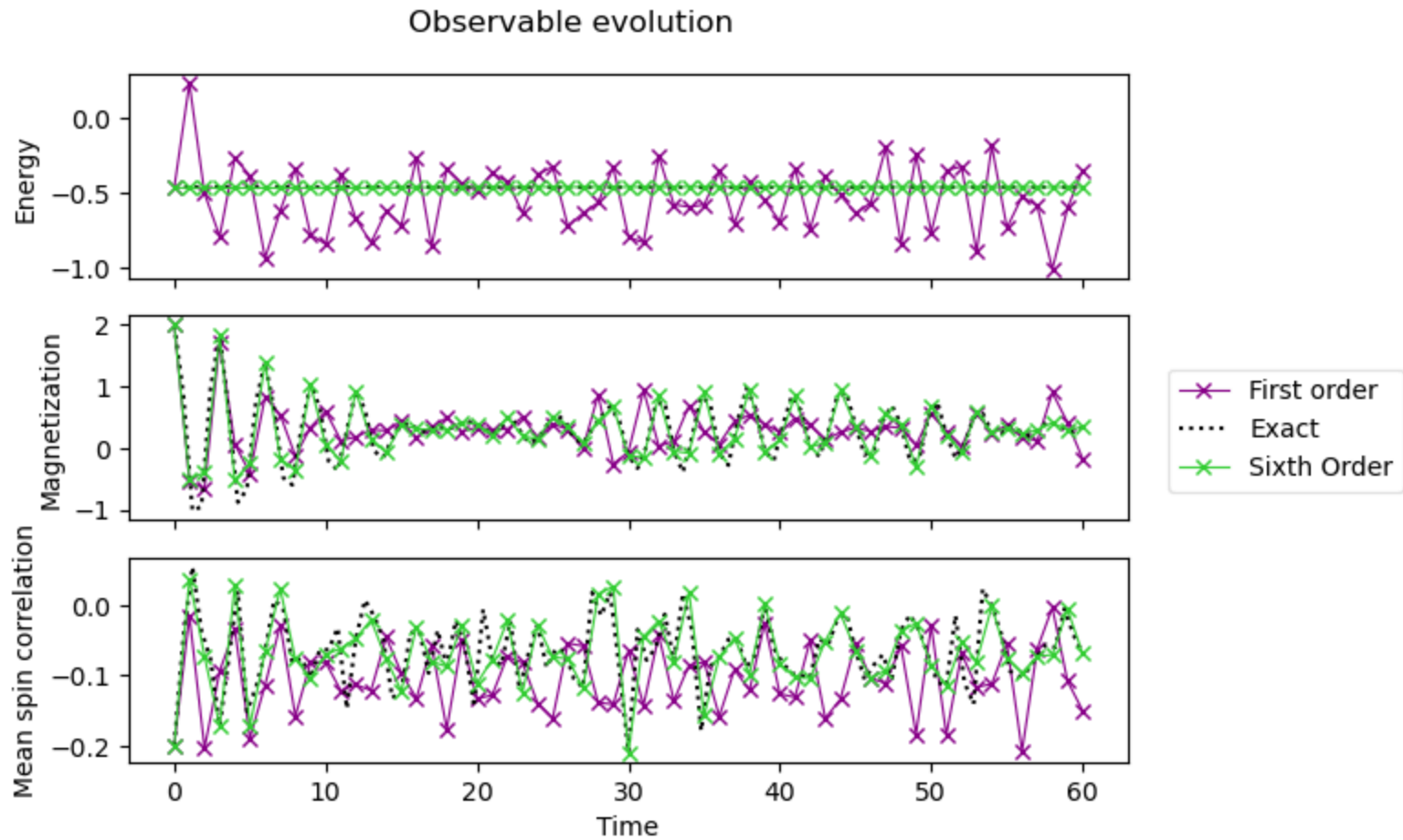
```

In [48]: `axes[0].plot(
 times, observables_order6[: , 0], label="Sixth Order", marker="x", c="limegreen", ls="--", lw=0.8
)
axes[1].plot(`

```
times, observables_order6[:, 1], label="Sixth Order", marker="x", c="limegreen", ls="-", lw=0.8
)
axes[2].plot(
    times, observables_order6[:, 2], label="Sixth Order", marker="x", c="limegreen", ls="-", lw=0.8
)

# Replace the Legend
legend.remove()
legend = fig.legend(
    *axes[0].get_legend_handles_labels(),
    bbox_to_anchor=(1.0, 0.5),
    loc="center left",
    framealpha=0.5,
)
fig
```

Out[48]:



```
In [50]: from matplotlib import cm

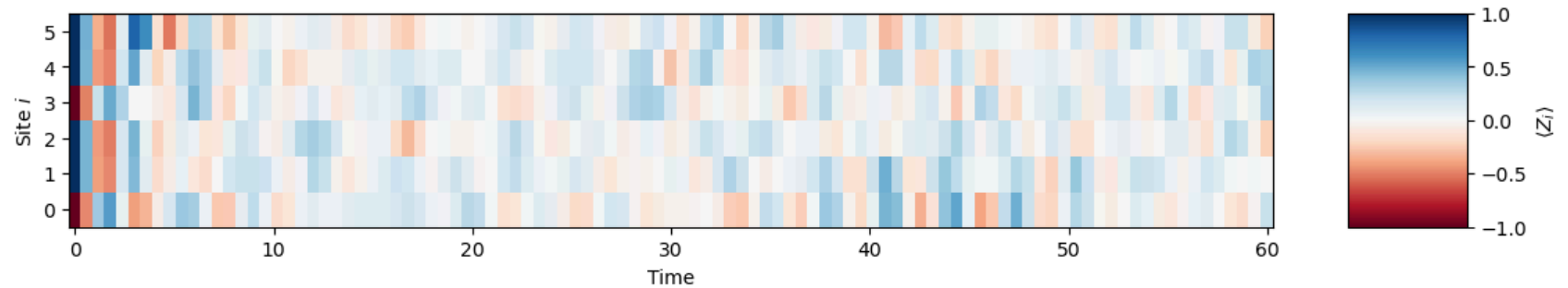
magnetizations = np.real(
    [[sv.expectation_value(term) for term in magnetization_op] for sv in exact_evolution]
)

plt.figure(figsize=(14, 2))
xx, yy = np.meshgrid(exact_times, np.arange(L))
plt.pcolor(xx, yy, magnetizations.T, vmin=-1, vmax=+1, cmap="RdBu")

plt.yticks(np.arange(L))
plt.ylabel("Site $i$")
```

```
plt.xlabel("Time")  
plt.colorbar(label="$\\langle Z_i \\rangle$", aspect=1.8)
```

Out[50]: <matplotlib.colorbar.Colorbar at 0x16ae0da50>



Indeed, the spins of the lattice sites become more uniform over time as the external field magnetizes the material.